

JARVIS AI

Voice-enabled Windows Desktop Assistant | Hackathon Project Report

Project Goal	A desktop assistant that accepts voice or text commands, secures access with face and pattern authentication, and helps users search, chat, remember information, open apps, and communicate faster.
Built With	Python, Eel, HTML/CSS/JavaScript, SQLite, OpenCV, Groq AI, Firebase (optional), OpenWeather, ADB and WhatsApp.
Presentation Line	Jarvis AI combines an attractive assistant interface with practical automation and privacy-focused local security.

Prepared for college hackathon submission

1. Project in Simple Words

Jarvis AI is a Windows desktop voice assistant. It works like a small personal helper: the user can speak or type a command, and Jarvis identifies the request, performs an action, and shows the answer in the interface.

The project uses a web-like front end inside a desktop application. This gives it a modern visual interface while Python handles the real system actions.

The key differentiator is the secure entry flow: face recognition is checked first, then the user unlocks the app using a 3x3 pattern lock.

2. Main Features

Voice input through microphone using SpeechRecognition.

Text chat input from the Jarvis interface.

Face authentication using OpenCV LBPH face recognizer.

Pattern lock with salted SHA-256 hash; the raw pattern is not stored.

AI chat through Groq using llama-3.3-70b-versatile.

Open installed applications or saved websites.

Play a requested song/video on YouTube.

Google search and live weather lookup.

Remember and recall short personal facts from SQLite.

WhatsApp messaging, WhatsApp call/video call, and Android mobile calling through ADB.

Chat sessions and history in local SQLite; optional Firebase Firestore cloud sync.

Modern animated UI, code-answer formatting and copy button for AI code replies.

3. How the Project Works

User opens Jarvis -> face camera verifies the user -> pattern lock opens -> user gives a voice or text command -> Python command router selects the right feature -> result is shown on screen and can be spoken aloud.

If no fixed command matches, the message goes to the Groq AI chatbot. The response is displayed and saved to the active chat session.

Input	Security	Command Router	Action / Output
Voice or text	Face + pattern	allCommands()	App, web, AI, weather, memory or communication

4. Code Explanation - File by File

run.py - Starts the project. It sets the correct working folder and calls **main.start()**.

main.py - Main desktop app setup. It starts Eel, opens Microsoft Edge in app mode, exposes pattern-lock functions to JavaScript, plays startup sound, and controls face-authentication flow.

engine/command.py - The command control center. **takecommand()** listens through microphone; **allCommands()** decides which feature should run; **speak()** sends messages to UI and optionally speaks them using Windows SAPI.

engine/feature.py - Contains actual actions: opening apps/sites, YouTube, Google, weather, memory, WhatsApp, ADB calls, and Groq AI chat.

engine/pattern_lock.py - Creates, verifies, changes and removes the secure pattern. A unique salt plus SHA-256 hash protects saved pattern data.

engine/auth/recogonize.py - Uses webcam, Haar Cascade face detection and trained LBPH model to verify the saved face.

cloud.py - Makes chat sessions, stores chats in **jarvis.db**, and optionally syncs unsynced records to Firebase Firestore.

www/ - Interface files. **index.html** gives page structure, **style.css** gives visual design, **main.js** sends microphone/text actions, **controller.js** controls UI and pattern lock, and **script.js** renders chat history and code blocks.

engine/db.py - SQLite table setup/demo database operations for saved commands, contacts and memory.

5. Important Code Logic to Explain to Judges

Command routing: `allCommands()` checks command keywords in order. For example, 'open chrome' calls `openCommand()`, 'weather in Delhi' calls the weather function, and unknown/general questions go to `chatBot()`.

AI fallback: Instead of creating separate rules for every question, general questions are sent to Groq. This makes Jarvis useful beyond fixed commands.

Local-first chat storage: Chat messages are always saved in SQLite. If Firebase credentials are available, the same records sync to cloud; if not, the app still works locally.

Security: The pattern is stored as a hash with a random salt. Face verification is used as the first gate before the pattern screen.

Frontend-backend bridge: Eel connects JavaScript and Python. JavaScript can call Python functions such as `eel.allCommands()`, and Python can update the UI using `eel.DisplayMessage()`.

6. Demo Script (2 to 3 Minutes)

Start with: 'This is Jarvis AI, a secure voice-enabled Windows desktop assistant.'

Show face recognition, then draw the pattern lock.

Type or speak: 'open calculator' or 'open youtube'.

Show AI chat: ask a normal question; then show a code question to demonstrate formatted code and Copy button.

Say: 'weather in Delhi' after setting OpenWeather API key.

Say: 'remember my college is [college name]', then ask 'what is my college'.

Open the chat panel and show previous sessions/history.

Close with benefit: 'Jarvis brings security, AI conversation, and common desktop/mobile actions into one interface.'

7. Setup Before Demo

Use Windows with a working webcam, microphone and Microsoft Edge.

Activate the Python environment and install packages: `pip install -r requirements.txt`.

Create a local `.env` file with `GROQ_API_KEY` and `OPENWEATHER_API_KEY`. Never upload this file.

For cloud history, add Firebase service-account file as `firebase_key.json`. This is optional.

For mobile calls, install ADB, enable USB/Wi-Fi debugging on Android, and connect the device first. This is optional.

Run `python run.py`. For an executable, run `build_app.bat`; output is expected at `dist/Jarvis/Jarvis.exe` after a successful PyInstaller build.

8. How to Submit the Project

Create one clean submission folder or GitHub repository. Include source code, README, requirements, presentation PDF, screenshots/video and setup instructions.

Include: main.py, run.py, engine/, www/, requirements.txt, README.md, FEATURES.md, build_app.bat and this PDF.

Do NOT upload: .env, firebase_key.json, pattern_lock.json, jarvis.db with personal data, face sample images/trainer if they contain private biometric data, virtual environment folder, and build/cache folders.

If submitting a GitHub link, add a .gitignore for secrets, local databases, Python virtual environment and generated output.

Attach a 1 to 3 minute demo video. It helps judges even if the live demo faces network, webcam or API issues.

For hackathon portal: upload project zip + PDF + demo video link + GitHub link if requested.

9. Suggested Submission Checklist

Test webcam, microphone, face model, pattern lock and internet before submission.

Keep a backup demo recording and 4 to 6 screenshots.

Add your name, team name, college and GitHub link to README/PDF if required.

Confirm API keys are present locally but not committed.

Explain optional modules honestly: Firebase/ADB need their own configuration.

If packaging fails, submit source code with exact run steps; do not claim the EXE is included unless it is tested.

10. Limitations and Future Scope

Face recognition uses a pre-trained local model and may need retraining for a different user. Voice recognition and cloud features depend on microphone/internet availability. WhatsApp/ADB automation depends on installed apps and connected Android device.

Future improvements: multi-user login, better intent classification, language selection, encrypted local database, reminders/calendar integration, robust error logs, and a packaged installer with first-run setup.

One-line conclusion: Jarvis AI is a practical desktop automation project that combines voice interaction, AI chat, local data, cloud-ready history and layered user security.